The Evolution of Modern CPU Architecture

Lecture Notes — Foundations of HPC

Course: Foundations of High-Performance Computing
Lecturer: Luca Tornatore — Università di Trieste
Academic Year: 2025/2026
Reference textbook: J. Hennessy & D. Patterson, *Computer Architecture: A Quantitative Approach*, 7th Edition, 2025

These notes (see the final notes about their origin) accompany the first lecture on Modern Architecture and are meant to serve as a self-contained reference for the topics covered. The intended audience are the students enrolled in the course that have attended the same lectures.

The exposition follows a chronological thread: we start from the idealized Von Neumann model, trace the forces that reshaped processor design over five decades, and arrive at the complex machines we program today.

---

1. Setting the Stage: Von Neumann vs. Today

1.1 The Classical Von Neumann Model

It is of fundamental importance to commence this analysis by delineating the profound departure of contemporary computational platforms from the theoretical foundations upon which computer science was originally built. It is widely recognized that the classical Von Neumann architecture, formulated in 1945, is still the conceptual backbone taught in every introductory course on computer science. Its assumptions are few and clean:

- There is **one processing unit** — a single ALU governed by a single control unit.
- **One instruction is executed at a time**, strictly in program order, and every operation has the same "cost".
- **Memory is flat**: access to any location costs the same, and that cost is comparable to performing an arithmetic operation.

It is worth pausing a moment to acknowledge the depth of this abstraction. Before Von Neumann, building a computer meant hard-wiring the algorithm into the machine; his stored-programme concept was a revolution. And the model still provides the correct mental framework for *reasoning about correctness* — sequential semantics, a single thread of control, a uniform address space. The point I want to raise is that it has become a dangerously misleading model for *reasoning about performance*.

1.2 A Modern Processor: What Has Changed

It is necessary to underline the fact that modern processor divergence has rendered these classical assumptions entirely obsolete and fundamentally inaccurate.
Take a not-particularly-extreme example: a 6-core Intel Xeon e5600, or any contemporary Skylake-generation chip. Against each Von Neumann assumption, the reality diverges sharply:

- There are **many processing units** — multiple cores, each one containing multiple functional units

(integer ALUs, floating-point units, load/store units, branch units, vector units).

- **Many instructions execute simultaneously**, both within a single core (instruction-level parallelism, ILP) and across cores (thread-level parallelism, TLP). Internally, high-level instructions are decomposed into simpler micro-operations (μops) that flow through pipelines and can be reordered.
- **Different instructions have very different costs.** A register-to-register addition may take 1 cycle; a floating-point division 14–20 cycles; a cache miss to DRAM easily 150–300 cycles.
- **Memory is profoundly non-flat.** The memory landscape in a modern computing node is decidedly non-flat. A deep hierarchy separates registers (~200 ps access) from L1 cache (~1 ns), L2 (~3–10 ns), L3 (~10–20 ns), main DRAM (~50–100 ns), and storage (milliseconds). Accessing DRAM is orders of magnitude more expensive than performing an arithmetic operation.

The currency in which all these "costs" are measured is the **CPU cycle**. This is the key metric — and one that the Von Neumann model does not even contemplate, since in that framework every operation is "1 step".

The gap between the naive model and physical reality is, in the end, the reason why this course exists. In the next sections we trace *how* and *why* the architecture evolved from Von Neumann's elegant simplicity to today's complexity, following the historical path through the key technological and physical turning points.

---

2. Moore's Law and the "Free Lunch" Era

2.1 Moore's Observation

In 1965, Gordon Moore predicted that the number of transistors on an integrated circuit would double every year. He revised this in 1975 to a doubling every two years. Let us be precise: this is not a physical law — it is an empirical observation about the pace at which the semiconductor industry managed to shrink transistor dimensions and increase integration density. Yet it held remarkably well for roughly fifty years, which is an almost absurd duration for an exponential trend in technology.

The mechanism is, at root, straightforward. Advances in lithographic techniques allowed smaller and smaller feature sizes on the silicon wafer. Smaller transistors mean more transistors per unit area — density scales quadratically with the linear shrink factor — and, as we shall see in Section 3.1, smaller transistors also switch faster. Feature sizes went from 10 μm in 1971 down to 7 nm in 2022, with 3 nm chips already in production and Intel announcing plans for 1.8 nm (which they have relabelled "18 Å", presumably to set the psychological stage for sub-nanometre marketing). The combined effect was a growth in transistor count on a chip of about 40–55% per year during the golden decades.

The scale of this growth is difficult to internalise. The Intel 80286 (1982) had 134,000 transistors. The Pentium (1993) had 3.1 million. The first Core i7 (2010) reached 1.17 billion. By 2022, the Apple M2 packed 20 billion transistors — though even this falls a factor of about 5 short of what a naive extrapolation of Moore's Law from 2010 would have predicted (roughly 106 billion). The message is clear: Moore's Law is slowing, and the doubling time is stretching with each new technology generation (Hennessy & Patterson, 7th ed., §1.4 and §1.12). Unlike in the classical Moore's Law era, we should expect the interval between doublings to keep increasing going forward.

2.2 Two Dividends of Transistor Scaling

During the decades when Moore's Law held in its strongest form, the growing transistor budget delivered two distinct performance dividends at the same time. It is important to separate them conceptually, because they broke down at different moments and for different reasons.

Dividend 1: More capable CPUs. More transistors funded more sophisticated on-chip logic. The progression is spectacularly visible in the Intel x86 lineage, which Hennessy & Patterson track in detail (Fig. 1.10, 7th ed.):

Year	Processor	Transistors	Key architectural step
1982	80286	134 K	16-bit address/bus, microcode
1985	80386	275 K	32-bit address/bus, microcoded
1989	80486	1.2 M	5-stage pipeline, on-chip I&D caches, integrated FPU
1993	Pentium	3.1 M	2-way superscalar, 64-bit bus
1997	Pentium Pro	5.5 M	Out-of-order, 3-way superscalar
2001	Pentium 4	42 M	OoO superpipelined (~31 stages), on-chip L2
2010	Core i7	1,170 M	Multicore OoO, 8 MB L3, Turbo mode
2021	Xeon	~billions	8 cores, 40 MB L3, Turbo to 5.1 GHz

Each generation exploited its larger transistor budget to add pipelines, caches, out-of-order logic, wider issue windows, branch predictors, vector units, and eventually multiple cores. The chip did not merely get "bigger" — it acquired qualitatively new capabilities at each step. The jump from the 80386 (microcoded, in-order) to the Pentium Pro (out-of-order, speculative, 3-way superscalar) is as profound a change in the internal organisation of the machine as going from a bicycle to a car.

Dividend 2: Higher frequency. Smaller transistors switch faster because the gate delay scales down with the feature size. To a first approximation, delay $\tau \sim RC \propto 1/\kappa$, where κ is the linear shrink factor. This fed directly into higher clock frequencies. The record, tracked by Hennessy & Patterson in Fig. 1.11 (7th ed.), shows:

- 5 MHz for the VAX-11/780 in 1978
- 25 MHz for the MIPS M2000 in 1988
- 500 MHz for the Alpha 21164A in 1996
- 1 GHz for the Pentium III in 2000
- Over 3.2 GHz for the Pentium 4 Xeon in 2003

The growth rate was roughly 15%/year before 1986, then accelerated to about **40%/year during the RISC renaissance** (1986–2003) — the same period in which architectural innovation was also at its peak. After 2003, as we shall discuss, it fell essentially to zero.

Higher frequency translates directly into more operations per second: even running the same code, on the same microarchitecture, a faster clock delivers proportionally more throughput. This required no effort whatsoever from the programmer.

2.3 The "Free Lunch"

The combination of these two dividends — more capable logic *and* faster clocks — created what Herb Sutter memorably called the **"free lunch" era** *(«free lunch» is an expression firstly introduced by D. Sutter in a classical paper on that. See references)*. Software got faster simply because the hardware it ran on got faster, generation after generation. A program compiled once would run measurably faster on the next year's CPU without changing a single line of code.

This was a remarkably comfortable arrangement, especially for the software side. Programmers could concentrate on correctness, maintainability, and higher-level abstractions, trusting that Moore's Law would keep delivering performance for free. And this trust was well founded: between 1986 and 2003, the combined effect of architectural innovation (RISC designs, pipelining, superscalar techniques) and semiconductor scaling produced a compound annual growth rate of about **52% in single-thread performance**, as measured by SPECint benchmarks (Hennessy & Patterson, Fig. 1.1). By 2003, the best microprocessors were roughly **7.5× faster** than what pure technology scaling alone (~35%/year) would have delivered — the additional factor came from architectural cleverness funded by the growing transistor budget.

But every exponential growth driven by a physical process eventually hits physical limits. And around 2003–2004, two of them hit essentially simultaneously.

---

3. The Power Wall and the End of Dennard Scaling

3.1 Dennard Scaling: Smaller and Faster at Constant Heat

In 1974, Robert Dennard and colleagues at IBM published a landmark paper describing what is now called **constant-field scaling** (or simply "Dennard scaling"). The argument is elegant and can be summarised quantitatively. Suppose one shrinks every linear dimension of a MOSFET by a factor $\kappa > 1$, so that the new gate length becomes $L' = L/\kappa$. If one simultaneously scales the supply voltage V_{DD} and the threshold voltage V_T by $1/\kappa$, and adjusts the doping concentrations to keep the electric field approximately constant, then:

- **Electric fields remain approximately constant**, since $E \sim V/L$ and both V and L scale by the same factor $1/\kappa$.
- **Gate capacitance scales down**: $C \propto \varepsilon A/t \sim 1/\kappa$, where A is the gate area ($\propto 1/\kappa^2$) and t is the oxide thickness ($\propto 1/\kappa$).
- **Switching delay decreases**: $\tau \sim RC \propto 1/\kappa$, so **frequency can go up** as $f \propto \kappa$.
- **Dynamic power per gate drops**: $P_{dyn} \sim CV^2f \propto (1/\kappa)(1/\kappa^2)(\kappa) = 1/\kappa^2$.
- **Transistor density increases** as κ^2.

Multiplying density (κ^2) by per-gate power ($1/\kappa^2$) gives us the crucial result: **power density ≈ constant**.

In a single phrase: *smaller, faster, at constant heat.* This was the magic that made the free lunch possible. One could pack more transistors, run them at a higher clock, and the chip did not get hotter. The consequences were profound: not only could you add complexity (more logic, bigger caches), but you could also push frequency up aggressively without worrying about thermal dissipation growing out of control.

3.2 The Dynamic Power Equation

The energy consumed per logic transition in CMOS is:

$$E_{\text{dynamic}} \propto C \cdot V^2$$

and the power (energy per unit time) is:

$$P_{\text{dynamic}} \propto C \cdot V^2 \cdot f$$

where C is the total switching capacitance, V is the supply voltage, and f is the clock frequency. As long as Dennard scaling held, the reductions in C and V compensated for the increase in f, and the total chip power grew only modestly despite the exponential rise in transistor count.

In practice, the first 32-bit microprocessors (like the 80386) consumed about 2 W. By the early 2000s, a high-performance Pentium 4 consumed around 100 W. Today, a server-class Intel Xeon Platinum 8380 at 2.8 GHz has a TDP of 270 W, and the latest generation of GPUs and HPC accelerators has crossed the 500–1000 W mark, requiring liquid cooling. The trajectory from ~2 W to ~100 W during 1985–2005 already pushed against the practical limit of what air cooling can handle for a silicon die that is roughly 2.5 cm on a side. After that, things got worse.

3.3 Why Dennard Scaling Broke

Around 2004, Dennard scaling ceased to hold. The breakdown was not a single dramatic event but rather the convergence of several physical limits, each of which independently would have been manageable but which together proved fatal.

1. The voltage floor. The supply voltage V_DD cannot be reduced indefinitely. The reason is that the threshold voltage V_T has a physical lower bound: for an ideal MOSFET at room temperature, the subthreshold slope bottoms out near ~60 mV/decade of current. Pushing V_T below a certain level causes the subthreshold leakage current (the current that flows even when the transistor is nominally "off") to grow exponentially. In practice, V_DD got stuck around ~0.7–1.0 V, and at that point the key energy term CV^2 simply stopped shrinking. This alone would have been enough to break the scaling.

2. Leakage currents. As gate oxides thinned to just a few atomic layers (1–2 nm of SiO_2 in early 2000s technology), quantum-mechanical tunnelling through the oxide became significant — electrons literally tunnel through the insulator. Add to this drain-induced barrier lowering (DIBL), which degrades the transistor's ability to turn fully off. The result is that **static (leakage) power**, P_static ∝ I_leak × V, became a first-order contribution to total dissipation. It is no longer the case that a chip consumes power only when switching; it now also "leaks" power continuously. In high-performance designs, leakage can easily account for 25–40% of total power — partly because of the enormous SRAM caches, which require constant power to hold their stored values.

3. Velocity saturation. At the electric field strengths encountered in channels shorter than ~100 nm, charge carriers hit velocity saturation — their drift velocity no longer increases linearly with the field. This means the ideal 1/κ improvement in gate delay no longer materialises in full; the actual delay shrinks less than Dennard predicted, and so the frequency benefit of each new technology node is smaller than expected.

4. The interconnect bottleneck. While transistors got faster with each generation, the wires that connect them did not cooperate. The signal propagation delay of a wire is proportional to its RC product. As wire cross-sections shrank, the resistance per unit length went up (worsened by surface and grain-boundary scattering of electrons in nanometre-scale copper lines), and the capacitance per unit length increased due to tighter spacing between adjacent wires. The net effect: **moving a signal across the chip can now be slower and**

costlier than switching a gate — a qualitative inversion of the historical balance. This is sometimes called the "wire crisis".

5. The thermal reality (the "power wall"). With voltage pinned but transistor counts and clocks still rising, power density began to increase instead of remaining constant. At a certain point, the chip simply cannot dissipate the heat it generates. Air cooling has a practical ceiling of roughly 100–300 W for a typical chip package. Had frequency continued to grow at the pre-2004 rate (~40%/year), a modern CPU would require the power density of a nuclear reactor core — which is obviously not viable.

3.4 The Consequences: Frequency Plateau and the Multi-Core Turn

The historical data leave no room for ambiguity. Hennessy & Patterson (Fig. 1.11, 7th ed.) track clock rate growth meticulously:

- **~15%/year** in the late 1970s to mid-1980s
- **~40%/year** from the mid-1980s to 2003 (the RISC renaissance)
- **~0%/year** from 2004 onward — base clock frequencies flat at 3–4 GHz

Intel has offered Turbo mode since 2008, where the chip temporarily boosts one or a few cores to a higher clock while others are idle or powered down. Even this mechanism has grown at a paltry **~2%/year** (from ~3.6 GHz Turbo in 2010 to ~5.1 GHz in 2021). Turbo mode is useful, but it is not exponential growth — it is a coping strategy.

The performance growth rates for single-thread execution tell the same story (Hennessy & Patterson, Fig. 1.1):

Period	Annual growth (SPECint)	Doubling time
1978–1986	~25%	~3.5 years
1986–2003	~52%	~1.5 years
2003–2011	~23%	~3.5 years
2011–2015	~12%	~6 years
2015–present	~9%	~8 years

The deceleration is dramatic. The 52%/year golden era is long gone; we are now in a regime where single-thread performance doubles roughly every 8 years. Note that since 2007, SPEC benchmarks enable automatic parallelisation, so even these modest numbers partly reflect multicore gains, not pure single-thread improvement.

In 2004, Intel cancelled its planned high-performance uniprocessor successors to the Pentium 4 (notably the Tejas project) and, together with the rest of the industry, declared that the path to higher performance would be through **multiple cores per chip** rather than faster single cores. As Hennessy & Patterson put it, this switch from relying solely on ILP to requiring DLP and TLP was a historic turning point.

The implications for programmers are severe:

- **Before ~2004:** performance came primarily from ILP within a single core, exploited implicitly by the hardware and the compiler. The programmer did not need to think about parallelism.
- **After ~2004:** continued scaling requires explicit parallelism — TLP (threads), DLP (vectors/SIMD), and

RLP (request-level parallelism in servers). These all demand that the programmer restructure the application.

This is where the "free lunch" ended. And this is, ultimately, why courses like this one exist.

3.5 Strategies to Cope with the Power Wall

Modern processors employ a battery of strategies to manage energy within their fixed thermal envelope:

Dark silicon. At any given moment, a significant fraction of the transistors on a modern chip must be powered off (power-gated) to stay within the thermal budget. Not all transistors can be active simultaneously — they are there to provide *capability* (e.g., specialised accelerators, large caches) that is used selectively, not all at once.

Dynamic Voltage and Frequency Scaling (DVFS). The chip can lower V and f during periods of low activity, trading performance for energy savings. Since $P \propto V^2 f$, even a modest reduction in voltage yields a quadratic reduction in dynamic power.

Turbo / boost modes. The inverse strategy: temporarily overclock one or a few cores while others are idle, staying within TDP on average.

Heterogeneous cores. Mixing high-performance ("big") and energy-efficient ("little") cores, dispatching work to the appropriate class. ARM big.LITTLE pioneered this in mobile; Apple adopted it for laptops and desktops.

Race to halt. Using a fast core to finish a job quickly, then entering a deep sleep state. The total energy can be lower than running a slow core for longer.

These are all manifestations of a fundamental shift in what is being optimised. The primary evaluation metric in processor design has moved from *performance per mm²* to **performance per watt** (or, equivalently, tasks per joule). This has profound implications for HPC, where the power bill of a large cluster is itself a significant operational cost.

---

4. The Memory Wall and the Rise of the Memory Hierarchy

4.1 The Processor-Memory Performance Gap

The second great challenge — historically preceding the power wall by about a decade — is the **memory wall**. Even if we had unlimited frequency and power, the speed of main memory (DRAM) would throttle the processor.

This problem originates from a severe historical mismatch in technological improvement rates between processing logic and storage mediums. Since the year 2000, while the computational processing speeds of CPUs and GPUs have skyrocketed by an astonishing factor of roughly 1000×, main memory bandwidth has grown by a mere 20× to 30×. More critically, the intrinsic access latency of the main memory has remained stubbornly stagnant, lingering perpetually in the range of 50 to 100 nanoseconds. This creates a scenario where a modern processor capable of executing billions of instructions per second spends the vast majority of its time completely stalled, waiting for data to arrive from the main memory.

The core of the problem is quantitative and well documented. Historical data show two trajectories diverging over time:

- CPU cycle times decreased at roughly **30%/year** during the high-growth era (1986–2003), corresponding to the ~52%/year single-thread performance improvement.

- DRAM **latency** improved at only about **7%/year** during the 1990s, slowing to below 1%/year more recently.

The data in the lecture slides (from a 2019 PhD thesis at UPC Barcelona, tracking CPU cycle time vs. DRAM access latency from 1990 to 2020) make this divergence visually striking: by around 1990, the CPU cycle time was already ~30× shorter than DRAM access latency; by 2020, the gap had widened to about **360×**. The CPU has got faster; the memory has barely budged.

A concrete snapshot: for a modern processor at ~3 GHz (cycle time ~0.33 ns):

Level	Typical latency	Approx. cycles	Typical size
Register	~200 ps	< 1	~100 B (per core)
L1 cache	~1 ns	~3–4	~32–64 KB (per core)
L2 cache	3–10 ns	~10–30	~256 KB – 2 MB
L3 cache	10–20 ns	~30–60	~8–40 MB (shared)
DRAM	50–100 ns	~150–300	~16–512 GB
SSD	~100 μs	~300,000	~TB
Disk	~5–10 ms	~15–30 million	~TB–PB

The span from register to disk covers roughly **eight orders of magnitude** in access time. This is the playing field on which all code runs.

4.2 Bandwidth vs. Latency: A Crucial Distinction

It is essential to distinguish between **bandwidth** (throughput — how much data per second can be transferred) and **latency** (response time — how long one must wait for the first byte of a single request). The two have evolved very differently.

Hennessy & Patterson (Fig. 1.9, 7th ed.) track both metrics across microprocessors, memory, networks, and disks over the past 25–45 years. The summary is striking:

- **Bandwidth** improved by factors of 450 to 80,000 across technologies.
- **Latency** improved by factors of only 5 to 64.

For memory specifically, bandwidth improved by a factor of about 4000 (from 13 MB/s for 16-bit DRAM modules in 1980 to 51 GB/s for DDR5 in 2021), while latency improved by only a factor of about 5 (from 225 ns to 46 ns). And even that modest factor is somewhat misleading, because the *ratio* of CPU speed to DRAM latency (which is what actually determines how many cycles the CPU wastes waiting) has got dramatically worse.

You may know that modern DDR4/DDR5 memory operates at multi-GHz clock rates (3–6 GHz). That sounds fast. But those high clocks buy you **bandwidth** — the ability to sustain a high transfer rate once a burst transfer is underway. They do not substantially help **latency**, which is the time from issuing a request to receiving the first useful datum. For small, random accesses — which is what happens when the CPU follows a pointer, or accesses an unpredictable array element — latency is what counts, and it has barely improved.

This bandwidth/latency asymmetry is a general technological trend (Patterson, 2004), and computer designers must plan accordingly. It is also why simply "adding more bandwidth" (wider buses, higher memory clocks) does not solve the memory wall — the fundamental bottleneck for many workloads is latency, not bandwidth.

4.3 Why Not Simply Make DRAM Faster?

A natural question that students always ask: why not just make DRAM faster? The answer lies in the physics and economics of the technology, and it is not encouraging.

DRAM stores each bit as charge in a tiny capacitor, accessed through a single transistor. This gives it excellent density and low cost per bit — one transistor per bit. But reading a DRAM cell requires sensing a very small charge difference, which is inherently slow (the sense amplifier needs time to resolve the tiny signal). The capacitor discharges during reading, so it must be restored (and periodically refreshed — hence "dynamic"), and the row/column access protocol adds multiple steps of latency.
The primary advantage of DRAM is its incredibly high density and low manufacturing cost, which seamlessly permits the creation of massive terabyte-scale capacities necessary for modern datasets.
However, this density comes at a profound, insurmountable performance cost.
The microscopic capacitor stores data as an electrical charge, but due to the physical imperfections of the dielectric materials, this charge inevitably leaks over time. As a consequence, the memory controller must continuously refresh the entire array (typically every 64 milliseconds), freezing access to the memory banks.
Furthermore, a read operation in DRAM is inherently destructive; the capacitor must be discharged into the bit-line to be sensed by the amplifier, and the data must subsequently be written back into the cell before another operation can occur.
These physical realities strictly guarantee that DRAM latency is constrained by fundamental analog circuit charging times. While bandwidth can be increased by widening the bus, the latency wall is dictated by physics and cannot be significantly improved; at best, it can only be mitigated.

SRAM (Static RAM), used for caches, stores each bit using six transistors arranged as cross-coupled inverters. This makes it intrinsically faster (no charge sensing, no refresh), but at the cost of ~6× the silicon area per bit and substantially more power. SRAM is simply too expensive for building a 64 GB main memory.

There is no known technology that combines DRAM's density and cost with SRAM's speed. Hennessy & Patterson are blunt about this: "a DRAM replacement technology remains unlikely" (§1.4, 7th ed.). The growth of DRAM capacity itself has dramatically slowed — from doubling roughly every 3 years historically, to a situation where the jump from 8 Gbit to 32 Gbit DRAM will take at least 10 years. Interestingly, the 24-Gbit DRAM shipped in 2023 was the first DRAM capacity that is not a power of 2, which says something about the difficulty of the scaling.

Memory Technology	Cell Architecture	Latency	Volatility & Refresh	Primary Application
SRAM	6T (Six Transistors), Flip-Flop	~1-3 ns	Volatile, No refresh required	CPU Caches (L1, L2, L3) 29
DRAM	1T1C (One Transistor, One Capacitor)	~50-100 ns	Volatile, Requires periodic refresh	Main System Memory 30

4.4 The Solution: A Hierarchy of Memories

Since we cannot make DRAM fast, the architectural response is to **bring small amounts of fast memory close to the CPU** and manage it automatically. This is the **memory hierarchy** — a cascade of memory levels of increasing size and decreasing speed, arranged so that the data most likely to be needed soon is kept in the fastest (and smallest) level.
The hierarchy, from fastest/smallest to slowest/largest:

1. **Registers** (~100 bytes, ~200 ps): inside the core, directly accessible by the ALU. The fastest storage, but extremely limited in number (typically 16–32 architectural general-purpose registers, plus floating-point and vector registers).
2. **L1 cache** (~64–128 KB, ~1 ns): split into separate instruction cache (L1i) and data cache (L1d), each typically 32 KB. Located inside each core, providing single-cycle access for hits.
3. **L2 cache** (~256 KB – 2 MB, ~3–10 ns): also typically per-core (sometimes shared by a small group of cores). Larger but slower than L1.
4. **L3 cache** (~8–40 MB or more, ~10–20 ns): shared among all cores on the chip. Serves as a last line of defense before going off-chip to DRAM.
5. **Main memory (DRAM)** (~64–512 GB, ~50–100 ns): large, but slow relative to the CPU. Every last-level cache miss incurs this penalty.
6. **Storage (SSD/disk)** (~TB–PB, ~100 µs for SSD, ~5–10 ms for disk): non-volatile, enormous, but astronomically slow relative to the CPU.

All cache levels are implemented in SRAM, fabricated on the same die as the processor (or, for some L3 caches, on the same package). The physical proximity is essential: the speed of light in silicon limits signal propagation to about 15 cm/ns, and wire delay is often the dominant factor for large caches. L1 is physically inside the core and can be accessed in a few cycles. L3, which is shared among all cores and may be tens of megabytes, sits further away and takes 30–60 cycles.

An important detail, often overlooked: **L1 is typically split into two separate caches** — one for instructions (L1i) and one for data (L1d), each usually 32 KB. This split allows the fetch pipeline and the load/store pipeline to access cache simultaneously without conflicts. L2 and L3 are unified (they hold both code and data).

The mere existence of this deep hierarchy fundamentally dictates how modern developers must write code. Software must be explicitly structured to maximize temporal locality (reusing data while it remains in the L1) and spatial locality (accessing contiguous memory addresses so the hardware prefetcher – *we'll see about prefetching in future lectures* – pulls the correct cache lines). For example, a two-dimensional matrix stored in memory in a row-major format must strictly be traversed sequentially by rows; failing to respect this physical layout will generate continuous cache misses, stalling the CPU to wait for DRAM and completely neutralizing the computational potential of the hardware. While novel interconnects such as Compute Express Link (CXL) and vertically stacked High Bandwidth Memory (HBM) have been introduced to expand aggregate bandwidth, the fundamental struggle against latency remains the paramount challenge of HPC engineering.

4.5 The Principle of Locality

The hierarchy would be useless if programmes accessed memory at random. Fortunately, real programmes exhibit a strong statistical regularity known as the **principle of locality**:

- **Temporal locality:** if a memory address is accessed now, it is likely to be accessed again in the near future. Think of a loop counter, a frequently-used variable, or a function that is called repeatedly.

- **Spatial locality:** if address *x* is accessed, nearby addresses *x+1*, *x+2*, etc. are likely to be accessed soon. Think of iterating sequentially over an array.

Caches exploit both. When a miss occurs, the hardware fetches not just the requested byte but an entire **cache line** (64 bytes on x86), betting on spatial locality. Once loaded, the data remains in the cache for a while, betting on temporal locality.

The effectiveness is remarkable: well-written programmes often achieve L1 hit rates above 95%, meaning that the vast majority of memory accesses are served in a few cycles instead of the hundreds that a DRAM access would cost. But "well-written" is the operative word here — poorly structured data access patterns can devastate cache performance.

4.6 The Quantitative Impact: Even Small Miss Rates Matter

Let us do a quick calculation. Suppose L1 access costs 2 cycles and a miss penalty (going to DRAM) is 100 cycles. With a 99% L1 hit rate:

$$\text{Average access time} = 0.99 \times 2 + 0.01 \times 100 = 1.98 + 1.0 = 2.98 \text{ cycles}$$

A mere 1% miss rate has increased the average memory access time by 50% compared to a perfect cache. At 5% misses, the average shoots to nearly 7 cycles — more than 3× the hit cost. At 10%, it is 12 cycles. This arithmetic explains why so much architectural effort (bigger caches, smarter prefetchers, more associativity) is devoted to shaving fractions of a percent off the miss rate.

There are three fundamental sources of cache misses — sometimes called the "three C's":

- **Compulsory misses** (cold misses): the first access to a given cache line always misses. Nothing can be done about this except prefetching.
- **Capacity misses**: the working set simply does not fit in the cache. The solution is either a larger cache or restructuring the algorithm to work on smaller blocks.
- **Conflict misses**: due to the limited associativity of the cache, two frequently-used addresses may map to the same cache set, evicting each other. Higher associativity helps but costs area and power.

4.7 The Energy Perspective

The energy cost of memory access reinforces the importance of the hierarchy in a way that is almost shocking. Data from TSMC's 7 nm technology node (reported by Hennessy & Patterson, Table 1.3, based on Jouppi et al., ISCA 2021) show:

Operation	Energy (pJ)	Relative to Int8 add
Int 8-bit add	0.007	1×
Int 32-bit add	0.030	4×
IEEE FP 32-bit add	0.380	54×
IEEE FP 32-bit multiply	1.310	187×
32-bit read, 8 KB SRAM	7.500	1,071×
32-bit read, 1 MB SRAM	14.000	2,000×

Operation	Energy (pJ)	Relative to Int8 add
32-bit read, DDR4 DRAM	1,300.000	185,714×

Reading a single 32-bit word from DRAM costs roughly **186,000×** the energy of an 8-bit integer addition, and about 93× the energy of reading from a 1 MB SRAM cache. Even reading from a small 8 KB SRAM (L1 scale) costs ~1,000× an integer add. The conclusion is unavoidable: **moving data is far more expensive than computing on it**, both in time and in energy. This is, in fact, the single most important quantitative fact for anyone who writes performance-critical code to internalise.

4.8 Implications for Programming

The memory hierarchy is not a hardware curiosity — it dictates how performant code must be structured. The key principles, which we will develop in depth in dedicated subsequent lectures, are:

- **Keep working data small**, so that it fits in the cache levels closest to the core. Prefer compact data structures. Every unnecessary byte in a struct is a byte that pollutes the cache.
- **Access data in contiguous, predictable patterns** — stride-1 iteration over arrays being the ideal — to exploit spatial locality and allow the hardware prefetcher to anticipate your accesses.
- **Reuse data while it is still in cache.** Arrange computations to work intensively on a block of data before moving to the next block (this is the principle behind loop tiling / blocking).
- **Minimise pointer-chasing and irregular access patterns**, which defeat both spatial locality and prefetching. Linked lists are the enemy of cache performance.
- **Be aware of the cache line size** (64 bytes on x86). Group related data within a cache line. In multi-threaded code, avoid false sharing (two threads writing to different variables that happen to share a cache line, forcing constant invalidation traffic).

---

5. Inside the Core: Frontend and Backend

5.1 The Two Halves

Internally, a modern CPU core is divided into two major sections with quite distinct responsibilities. The simplest way to think about it: the **frontend** is the orchestrator, and the **backend** is the execution engine.

The Frontend is responsible for figuring out *what* to execute:

- **Fetching** instructions from the L1 instruction cache.
- **Predecoding** and **decoding** the ISA-level instructions (e.g., x86 variable-length instructions, which are rather complex to parse) into simpler, fixed-format internal micro-operations (µops).
- **Branch prediction**: anticipating the direction of conditional branches *before* they are resolved, so that the fetch pipeline does not stall waiting for branch outcomes.
- **Dispatching** the decoded µops to the backend.

On a Skylake-class core (just an example), the frontend path looks roughly like:

> 32 KB L1 I-cache → Predecode → Instruction Queue → Decoders (5 wide) → µop cache (up to 6 µops/cycle) → µop queue → Dispatch (4 µops/cycle to backend)

Two structures deserve a mention. The **branch prediction unit** runs in parallel with the rest of the frontend, continuously feeding predicted next-addresses to the fetch logic so that it can proceed speculatively. And the **micro-op cache** (sometimes called the decoded stream buffer, DSB, or the "LSD" in Intel documentation) stores recently decoded µops, so that hot loops — which are exactly the loops we care about for performance — can bypass the full, slow x86 decode pipeline. This is a significant win: x86 decoding is expensive, and the µop cache effectively amortises that cost.

The Backend is responsible for actually *executing* the µops:

- **Register renaming** (allocating physical registers, eliminating false dependencies — more on this in Section 8).
- **Scheduling** µops for out-of-order execution, dispatching them to functional units as soon as their inputs are available.
- **Executing** operations on the multiple functional units (ALUs, FPUs, vector units, load/store units, branch resolution units).
- **Retiring** (committing) results in original programme order, so that the architectural state always reflects a valid sequential execution.

The boundary between frontend and backend is the **allocate/rename/retire** stage: µops arrive from the frontend in programme order; the backend takes responsibility for reordering their execution while guaranteeing correctness upon retirement.

5.2 The Scheduler and the Execution Ports

In the backend, a **scheduler** (also called the reservation station) holds a pool of µops waiting to execute. When all source operands of a µop become available, the scheduler dispatches it to a free functional unit through one of several **execution ports**.

On Skylake there are 8 ports. The precise mapping (which, for completeness, you can find in Intel's optimisation manuals and in Agner Fog's tables) is roughly:

- **Ports 0, 1, 5, 6**: arithmetic and logic (integer ALU, shifts, LEA, branches). Ports 0 and 1 also handle floating-point and vector FMA (fused multiply-add). Port 0 additionally handles integer multiplication and certain vector shuffle operations.
- **Port 4**: store data.
- **Ports 2, 3**: load and store-address generation.
- **Port 7**: store-address generation.

The critical point is this: the existence of **multiple ports** is what makes the core **superscalar** — in a single clock cycle, several ports can be active simultaneously, executing different µops in parallel, provided the µops are independent and the required port is free. This is the hardware foundation for ILP.

---

6. Superscalar Execution

6.1 Breaking the IPC = 1 Barrier

In a simple single-issue, in-order processor, at most one instruction completes per cycle: IPC ≤ 1. A **superscalar** processor breaks this ceiling by replicating functional units and adding the dispatch logic to issue N > 1 instructions per cycle. The idea first appeared commercially in the Intel Pentium (1993), which was 2-way superscalar.

The key requirement is straightforward to state: you need **multiple independent execution units** (ALUs, FPUs, load/store units, branch units) fed by a frontend that can fetch, decode, and dispatch N > 1 instructions per cycle. On Skylake, the effective **issue width** W — the number of μops that can begin execution simultaneously — is 6–8 (the number of ports that can fire in parallel).

The key architectural features of a superscalar design are:

- **Multiple independent execution units**, each capable of performing a different class of operation (integer arithmetic, floating-point math, memory load/store, branch resolution, vector/SIMD operations).
- A **frontend capable of fetching, decoding, and dispatching N instructions per cycle**. On Skylake, the decode width is 5 μops/cycle from the decoders (or 6 from the μop cache), and the dispatch width to the backend is 4 μops/cycle from the μop queue.
- **Issue width W**: the maximum number of μops that can begin execution in a single cycle. For a typical modern x86 core, W is effectively 6–8 (the number of execution ports that can be active simultaneously).

The theoretical peak IPC is W. In practice, achieving anything close to this requires that the instruction stream provides enough independent operations to fill all ports, that operands are ready, and that no stalls occur. Real-world sustained IPC on typical server workloads is often in the range 1–3, well below the theoretical peak. But for well-optimised, compute-bound inner loops, one can approach 4–6 — and that is where the performance payoff lies.

6.2 What Limits Superscalar Throughput?

Several factors prevent the machine from reaching peak IPC:

- **True data dependencies**: if instruction B reads a value produced by instruction A, B must wait for A to complete. This is fundamental — it reflects the actual logic of the computation and cannot be wished away.
- **Limited ILP in the instruction stream**: not all code naturally contains many independent operations within a small window. Long dependency chains (e.g., a series of dependent floating-point multiplications) or pointer-chasing patterns starve the machine.
- **Resource conflicts**: two μops targeting the same port must be serialised.
- **Branch mispredictions**: a misprediction flushes the pipeline and wastes all speculative work.
- **Cache misses**: a load miss stalls the dependent chain for tens to hundreds of cycles.

Addressing these limitations is precisely the job of pipelining, out-of-order execution, and register renaming. Superscalarity provides the *width*; the other mechanisms work to ensure that this width is utilised.

7. Instruction Pipelining

7.1 The Concept: Overlapping Instruction Stages

Until roughly the mid-1980s, it was natural to think of an "instruction" as an atomic operation that the CPU performs as a whole. Indeed, this was true for microcoded processors up to the 80386: the control unit would step through a sequence of microcode states for each instruction, and the next instruction would begin only after the previous one had completed.

But in fact, executing an instruction involves several logically independent steps, each using different hardware resources:

1. **Fetch**: retrieve the instruction from the instruction cache.
2. **Decode**: interpret it — identify the operation, source and destination registers, immediate values.
3. **Execute**: perform the computation (arithmetic, address calculation, etc.).
4. **Writeback**: store the result in the destination register or memory.

The insight behind pipelining is simple and powerful: while instruction I_1 is in the Execute stage, instruction I_2 can be in the Decode stage, and I_3 can be in the Fetch stage — simultaneously, because they use different hardware. This is the assembly-line analogy: each station works on a different item at the same time, and although any single item takes the same time to traverse the full line, the *rate at which finished items emerge* is much higher. The Intel 80486 (1989) introduced a 5-stage pipeline to the x86 world, and every processor since has been pipelined.

7.2 Pipelining Is About Throughput, Not Latency

This is a point that deserves emphasis, because it is often misunderstood. Pipelining does **not** reduce the latency of a single instruction — if anything, the added logic for pipeline registers and inter-stage handshaking slightly *increases* it. What pipelining improves is **throughput**: the rate at which completed instructions emerge from the pipeline.

A concrete example. Suppose an unpipelined processor executes each instruction in 500 ps (five stages of 100 ps each, done sequentially). The throughput is 1 instruction / 500 ps = 2 GIPS (giga-instructions per second). Now pipeline it into 10 stages of 50 ps each. After the pipeline fills (a startup cost of 10 × 50 ps = 500 ps), one instruction completes every 50 ps — a throughput of **20 GIPS**. The latency of a single instruction is still ~500 ps (actually a bit more, due to pipeline register overhead), but the sustained throughput is 10× higher.

In the ideal case, a k-stage pipeline delivers one instruction retired per cycle (IPC = 1), regardless of k. The pipeline depth determines the minimum achievable cycle time (and therefore the maximum clock frequency), while throughput is limited by the issue rate.

7.3 Superpipelining

Superpipelining means further subdividing pipeline stages into finer steps, increasing the depth k. A deeper pipeline allows a higher clock frequency, since each stage does less work and completes in less time. The tradeoff is:

- Higher **branch misprediction penalty** (more stages must be flushed — all work in the pipeline is wasted).
- Increased complexity in data forwarding between stages.

- More pipeline registers, more power consumption.

Modern x86 cores typically have 14–20+ stages. The Intel Pentium 4 (NetBurst architecture, 2001) pushed this to an extreme: approximately 31 stages, specifically to reach very high clock frequencies (up to 3.8 GHz at the time). But the misprediction penalty was devastating — roughly 20+ wasted cycles per bad branch — and the approach was ultimately abandoned. The industry returned to shorter, wider pipelines (more execution units at a moderate clock) with the Pentium M / Core architecture.

7.4 Pipeline Hazards

A pipeline achieves its ideal throughput only if a new instruction enters every cycle without interruption. In practice, three types of **hazards** disrupt this:

Data hazards arise from dependencies between instructions. They come in three flavours, traditionally classified by the order of reads (R) and writes (W):

- **RAW (Read After Write)**: instruction B reads a register that instruction A writes. B must wait for A to produce its result. This is a **true dependency** — it reflects a genuine data-flow relationship and cannot be eliminated (though it can be mitigated by forwarding, where the result of A is sent directly to B's input without waiting for the writeback stage).
- **WAW (Write After Write)**: two instructions write to the same register. In an in-order pipeline, the second must occur after the first. This is a **false dependency** — it arises from reuse of register names, not from any actual data flow. Register renaming (Section 8) eliminates it.
- **WAR (Write After Read)**: instruction B writes a register that instruction A reads. B must not overwrite before A has read. Also a false dependency, also eliminated by renaming.

Control hazards arise from branches. When the pipeline encounters a conditional branch, it does not know which instruction to fetch next until the branch condition is evaluated — which may happen many stages into the pipeline. The options are:

- **Stall** the pipeline until the branch resolves. This wastes cycles equal to the pipeline depth (or at least to the number of stages between fetch and branch resolution).
- **Predict** the branch outcome and fetch speculatively. If correct, zero cycles are wasted. If wrong, all speculatively fetched work must be flushed — a **misprediction penalty** of typically 15–20 cycles on modern cores.

Modern branch predictors achieve remarkable accuracy — 95–99% on many workloads — using sophisticated structures (branch target buffers, pattern history tables, TAGE predictors). But even 95% accuracy is not enough to be comfortable: in a tight loop with a branch every 5–7 instructions, a 5% misprediction rate means a flush roughly every 100–140 instructions, costing ~15–20 cycles each time. Over a long execution, this adds up.

Structural hazards occur when two instructions need the same hardware resource simultaneously. Superscalar designs mitigate this by duplicating functional units, but conflicts still arise when demand is concentrated on particular ports.

7.5 Branches: The Principal Enemy of Pipelining

Among all hazards, **control hazards from branches** are the most damaging to pipeline throughput. This is because:

- Branches are frequent: roughly 1 in every 5–7 instructions in typical compiled code is a branch.
- A misprediction flushes the entire pipeline, wasting all speculatively issued work.
- The cost grows linearly with pipeline depth.

The branch predictor is therefore one of the most critical — and one of the most complex — components of the frontend. Modern predictors use multi-level tables, global and local history patterns, and even techniques inspired by neural networks (perceptron predictors). But no predictor can be perfect, especially for inherently unpredictable branches (e.g., `if (data[i] > threshold)` where data is effectively random).

Implication for programming: minimising unpredictable branches in hot loops is one of the most effective optimisation strategies. Techniques include replacing `if/else` with branchless code (conditional moves, `cmov`), using arithmetic masks to select between values, restructuring data to make branch patterns more regular, and rewriting algorithms to avoid conditionals altogether. We will cover these in detail in the optimisation lectures.

---

8. Out-of-Order Execution

8.1 The Problem: In-Order Stalls

Consider a simple in-order superscalar processor trying to execute these four instructions:

```
I1: LOAD  R1, [addr1]     ; may miss in cache — 100+ cycles
I2: ADD   R2, R1, R3      ; depends on R1 from I1 — must wait
I3: MUL   R4, R5, R6      ; independent of I1, I2
I4: ADD   R7, R8, R9      ; independent of I1, I2, I3
```

An in-order processor executes these strictly in sequence. When I1 misses in cache, the processor stalls at I2 (waiting for R1), and cannot proceed to I3 or I4, even though they are completely independent of the stalled load. In a superscalar machine with 4–8 functional units, *all of them* sit idle for potentially hundreds of cycles — an enormous waste.

This is not a contrived example. Cache misses are common, and in real code, the instructions following a load are often (but not always) dependent on its result. An in-order machine pays the full latency penalty every time, even when independent work is available right there in the instruction stream.

8.2 The Idea: Execute When Ready, Not When Ordered

Out-of-order (OoO) execution decouples programme order from execution order. The principle is: an instruction executes as soon as all its input operands are available and a suitable functional unit is free, regardless of where it sits in the original programme sequence.

In the example above, an OoO processor would:

1. Issue I1 (the load) and send it to the memory pipeline.
2. Recognise that I2 depends on the result of I1, and park it in a waiting queue.

3. Issue I3 and I4 immediately — their operands are all available.
4. When I1's data finally arrives from memory, issue I2.

The key insight: I3 and I4 execute "for free" during the time that would otherwise have been dead cycles. The CPU has extracted **instruction-level parallelism** from the code by dynamically reordering execution at runtime.

8.3 The Mechanism: how OoO works

The OoO engine relies on several cooperating hardware structures:

The Reorder Buffer (ROB). As instructions are decoded by the frontend, they are placed into the ROB in programme order. Each ROB entry tracks the instruction's status: waiting, executing, or completed. The ROB enforces **in-order retirement** — results become architecturally visible (committed to the register file, effects on memory finalised) strictly in programme order, even though execution was reordered. This is essential for correct exception handling: if instruction I5 causes a page fault, the architectural state must reflect all instructions before I5 and none after it, regardless of what has been speculatively executed. On Skylake, the ROB holds approximately 224 entries — this is the "look-ahead window" within which the OoO engine searches for independent work.

The Reservation Station (RS) / Scheduler. Decoded µops are placed in the RS, each waiting for its source operands. When both operands are ready (either from the register file or via forwarding from a completing unit), the scheduler marks the µop as ready and dispatches it to a free execution port. The RS on Skylake has about 97 entries.

Register Renaming. This is perhaps the most elegant piece of the machinery. The processor maintains a pool of **physical registers** that is much larger than the set of **architectural registers** visible to the programmer (16 general-purpose registers in x86-64, for instance). When an instruction writes to an architectural register, the rename unit maps that write to a fresh physical register.
This means:

- Two instructions that both write to (say) architectural register RAX are mapped to *different* physical registers, eliminating the WAW conflict entirely.
- A read from RAX always refers to the physical register that was assigned by the most recent preceding write, which the rename table tracks.
- WAR hazards vanish because the "old" value lives in a different physical register from the "new" value.

Only **true dependencies** (RAW — where a real data-flow relationship exists) remain as actual execution constraints. This dramatically increases the effective ILP that the scheduler can exploit.

Why does this matter? Consider:

```
I1: ADD  R1, R2, R3    ; writes R1
I2: MUL  R4, R1, R5    ; reads R1 (RAW dependency on I1 — true)
I3: ADD  R1, R6, R7    ; writes R1 again — WAW with I1, WAR with I2
```

Without renaming, I3 cannot write to R1 until I2 has read the "old" R1 (WAR hazard), and the final value of R1 must come from I3, not I1 (WAW hazard). These hazards are *artificial* — they arise from the reuse of the name "R1", not from any genuine data-flow relationship.

With renaming, the hardware maps I1's write to physical register P17, I2's read from R1 to P17, and I3's write to a *different* physical register P23. Now there are no conflicts: I3 can execute as soon as R6 and R7 are ready, without waiting for I2 to finish reading. Only the **true dependency** (I2 must wait for I1) remains.

This dramatically increases the effective ILP that the scheduler can exploit.

8.4 OoO as Spatial Reordering

To crystallise the distinction between the two main forms of parallelism within a core:

- **Pipelining** is *temporal* overlap: different stages of different instructions execute simultaneously, progressing through the pipeline like items on an assembly line.
- **Out-of-order execution** is *spatial* reordering: instructions are dispatched to functional units based on operand readiness and resource availability, not based on their position in the programme.

The two are complementary and work together. Pipelining provides the throughput backbone (approaching IPC → 1 per pipeline). OoO ensures that the multiple functional units (superscalarity, IPC → W) are kept busy by finding independent work even when some chains are stalled on long-latency operations.

8.5 Historical Note: Tomasulo's Algorithm

The conceptual foundation for OoO was laid by Robert Tomasulo at IBM in 1967, for the floating-point unit of the IBM System/360 Model 91. His algorithm introduced reservation stations and a form of register renaming (using "tags" to track which functional unit would produce each pending result). This was remarkably ahead of its time.

What is striking is the gap between the idea and its widespread deployment. Tomasulo published in 1967; the first mainstream commercial OoO processors (Intel Pentium Pro, MIPS R10000, DEC Alpha 21264) did not appear until the mid-to-late 1990s — a gap of nearly 30 years. The reason is simple: implementing full OoO for all instructions, with a large reorder buffer and hundreds of physical registers, requires an enormous transistor budget. Tomasulo's idea had to wait for Moore's Law to provide the silicon.

This pattern — brilliant architectural ideas conceived decades before the technology makes them practical — is characteristic of the field, and it is one of the reasons why understanding the architecture historically, not just as a snapshot, gives a deeper appreciation of the design choices.

---

9. Putting It All Together

9.1 The Complementary Roles

It is worth stepping back to see how pipelining, superscalarity, register renaming, and out-of-order execution address distinct but interlocking bottlenecks. The summary table from the lecture slides captures this neatly:

Technology	Problem addressed	Improvement
Pipelining	Sequential execution bottleneck	Throughput → 1 IPC, disentangled from latency
Superscalarity	IPC = 1 ceiling	IPC up to W (the issue width)

Technology	Problem addressed	Improvement
Register renaming	False dependencies (WAR, WAW)	Removes name hazards, exposes more true ILP
OoO execution	In-order stalls from cache misses and long-latency ops	Exploits ILP dynamically, fills superscalar width
SMT (hyper-threading)	Single thread cannot fill all slots	Fills empty pipeline slots with µops from another thread

The logical progression is:

1. **Pipelining** overlaps instruction stages into stages and overlaps them. The aim is achieving a throughput of 1 IPC (1 Instruction per Cycle is however a cap).
2. **Superscalarity** adds multiple parallel pipelined execution units, raising the ceiling to W IPC — but a single thread may not provide W independent operations per cycle.
3. **Register renaming** strips away the artificial WAW/WAR dependencies, exposing the true data-flow graph so the scheduler sees more independent work.
4. **OoO execution** reorders instructions dynamically to exploit whatever parallelism the data-flow graph offers (ILP, Instruction Level Parallelism), filling the superscalar width as much as the true dependencies in the code allow.
5. **SMT (simultaneous multi-threading)** acknowledges that even OoO cannot always saturate all units from a single thread (due to inherently limited ILP, cache misses, or branch stalls), and interleaves µops from a second hardware thread to fill the remaining gaps.

Each layer addresses a limitation that the previous layer could not resolve. Together, they allow a modern core to achieve sustained IPC of 2–4 on well-optimised code, even though the theoretical peak W may be 6–8.

9.2 A Concrete Example: The Life of an Instruction on Skylake

Let us trace an instruction through a Skylake core, just to make the abstractions concrete:

1. **Fetch:** the branch predictor supplies a predicted address. The L1 instruction cache delivers up to 16 bytes of x86 instructions per cycle. The predecoder identifies instruction boundaries (x86 is variable-length: 1–15 bytes per instruction, which makes boundary detection non-trivial).
2. **Decode:** up to 5 decoders translate x86 instructions into 1–4 µops each. Alternatively, the µop cache supplies up to 6 µops/cycle for recently decoded instructions. Hot loops almost always run from the µop cache.
3. **Allocate/Rename:** the rename unit maps architectural registers to physical registers. µops are assigned entries in the ROB (~224 entries) and the reservation station (~97 entries). From this point on, programme order is maintained only by the ROB; the RS treats µops purely by data readiness.
4. **Schedule/Dispatch:** each µop in the RS waits for its source operands. The scheduler picks ready µops and dispatches them to the appropriate execution ports — up to 8 per cycle.
5. **Execute:** the functional unit performs the operation. Latencies vary: integer add = 1 cycle, FP multiply = 4 cycles, FP divide = ~14–20 cycles, L1 load = ~4 cycles. Results are forwarded to dependent µops immediately.
6. **Retire/Commit:** the ROB retires µops in programme order. When the oldest entry has completed and

all prior entries have retired, its result is committed to the architectural state. Only at retirement does the instruction become "officially" done.

The entire in-flight instruction window — the number of μops simultaneously in various stages between allocation and retirement — is bounded by the ROB size (224 on Skylake). This is the horizon within which the OoO engine searches for independent work to fill the superscalar width. A larger ROB means the engine can "see further ahead" in the instruction stream and tolerate longer latencies (such as L2/L3 cache misses).

---

10. A Synthetic Historical View

To anchor the entire preceding discussion, here is a condensed chronological overview of CPU architecture evolution, mapping hardware innovations to the physical challenges that motivated them. This follows the timeline presented in the course slides.

10.1 1950s–1970s: The Simple Era

Architectures were comparatively simple, and — crucially — memory latency was not far from CPU operation latency. Frequency increase was the primary driver of performance improvement. Tomasulo's algorithm (1967) was conceived at IBM for the System/360 Model 91 but not yet practical for general deployment: the transistor budget was simply too small. The Von Neumann abstraction was, at this point, a reasonably accurate description of actual hardware.

10.2 1980s–1990s: Superscalarity and OoO

The growing transistor density (Moore's Law at full steam) funded qualitatively new on-chip logic:

- **Pipelining** became standard (Intel 80486, 1989: 5-stage pipeline).
- **Superscalar execution** appeared (Pentium, 1993: 2-way superscalar).
- **Out-of-order execution** was deployed (Pentium Pro, 1997; MIPS R10000; Alpha 21264).

ILP techniques became the primary source of performance beyond raw clock speed. This was the peak of the "free lunch" era, with a compound ~52%/year growth in single-thread performance.

10.3 1990s–2000s: The Memory Wall

CPU speed outstripped DRAM latency by orders of magnitude. The response:

- **Deep cache hierarchies** (L1 → L2 → L3) became indispensable.
- **TLBs** (Translation Lookaside Buffers) were critical for virtual memory performance — a TLB miss can cost ~100 cycles.
- **Hardware prefetchers** attempted to load data into cache before the CPU requested it.
- **Branch predictors** grew in sophistication and importance.
- **SIMD** (MMX 1997, SSE 1999, SSE2 2001) appeared, adding data-level parallelism.

The memory wall was the first crack in the free lunch: programmers could no longer ignore data layout and access patterns without paying a severe performance penalty.

10.4 Mid-2000s: The Power Wall and Multi-Core

Dennard scaling ended. Frequency flattened at 3–4 GHz. The power wall was hit.

- **Multi-core** processors became the standard response: Core 2 Duo (2006), quad-core (2007), 8+ cores in the 2010s.
- Thread-level parallelism (TLP) became the primary path for continued scaling.
- New challenges appeared: **cache coherency** (maintaining consistency across private caches of different cores), **memory models** (what ordering guarantees does the hardware provide for shared-memory accesses), **NUMA** (non-uniform memory access latency depending on which core and which memory bank).
- The burden of exploiting parallelism shifted decisively to the programmer.

10.5 2010s–2020s: The Cambrian Explosion

With Moore's Law decelerating and Dennard scaling dead, the industry diversified:

- **Wider vectors:** AVX (256-bit, 2011), AVX-512 (512-bit, 2017) on x86.
- **Heterogeneous computing:** GPUs repurposed for general computation (GPGPU), TPUs for machine learning, FPGAs for specialised workloads. SIMD on CPUs, SIMT (Single Instruction Multiple Threads) on GPUs.
- **High-Bandwidth Memory (HBM):** very expensive stacked DRAM, designed to deliver extreme bandwidth for GPU and HPC accelerator workloads, at the cost of even higher latency than standard DRAM.
- **Chiplets and advanced packaging:** instead of monolithic dies, processors assembled from smaller chiplets connected by high-speed on-package interconnects (AMD EPYC with Infinity Fabric, Intel Ponte Vecchio). This sidesteps yield and reticle-size limits.
- **High-speed interconnects:** NVLink (NVIDIA), Infinity Fabric (AMD), CXL (Compute Express Link) — all addressing the need for tighter coupling between processors, accelerators, and memory.

10.6 What Next?

Speculative, but the directions under active investigation include: near-data and in-memory computing (moving the computation to where the data resides, rather than the reverse), domain-specific architectures and ISAs (as GPUs and TPUs already are), and neuromorphic computing (a radically different paradigm inspired by biological neural networks, where the distinction between logic and memory is blurred). Whether any of these will deliver the universal, sustained improvement that Moore's Law provided for fifty years is, to put it gently, an open question.

---

11. Summary: Why All of This Matters When You Write Code

Every architectural feature discussed in these notes has direct consequences for how one writes performance-critical code. The detailed techniques belong to the subsequent lectures, but the high-level map is worth laying out now:

The memory hierarchy means that data locality is the single most important factor for performance. How one organises data in memory — array-of-structures vs. structure-of-arrays, loop tiling, blocking, cache-oblivious algorithms — is often more important than the choice of algorithm itself (at least for algorithms of the same asymptotic complexity). In HPC, "feed the CPU" is the first commandment.

Superscalarity means that the code must expose enough independent operations to keep the multiple execution units busy. Long dependency chains (e.g., an accumulation loop with a single running sum) starve the machine. Breaking the dependency chain — for instance by using multiple independent accumulators — can double or triple throughput at zero algorithmic cost.

Out-of-order execution means that the hardware will try to find parallelism for you, but it can only search within its instruction window (the ROB size). If all instructions in the window depend on a single stalled load, OoO cannot help. Writing code that provides many independent instruction chains gives the OoO engine the room to manoeuvre.

Pipelining means that **throughput, not latency, is the relevant metric** for sustained performance. An operation that takes 4 cycles of latency but accepts a new input every cycle (i.e. it is fully pipelined) achieves an effective throughput of 1 result per cycle in steady state. When reasoning about the performance of an inner loop, one should think in terms of throughput, not in terms of the latency of individual operations.

Branch prediction and pipeline depth mean that unpredictable branches are expensive. Every misprediction costs ~15–20 cycles of wasted work. In hot loops, replacing conditionals with branchless code (conditional moves, arithmetic masks, `cmov`) can yield substantial speedups.

Multi-core means that sequential code will not get faster as processor generations advance. At ~9%/year single-thread improvement, the performance of a serial programme doubles only every 8 years. Exploiting multiple cores requires explicit parallel programming (OpenMP, MPI, or other paradigms), which in turn demands understanding of thread safety, synchronisation costs, cache coherency overhead, and load balancing. These are the subjects of the lectures that follow.

The free lunch is over. The hardware still provides extraordinary capabilities — but accessing them requires understanding the machine. That is, ultimately, what this course is for.

References

1. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 7th Edition, Morgan Kaufmann, 2025.
2. R. H. Dennard, F. H. Gaensslen, H.-N. Yu, V. L. Rideout, E. Bassous, and A. R. LeBlanc, "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions," *IEEE J. Solid-State Circuits*, vol. SC-9, no. 5, pp. 256–268, 1974.
3. G. E. Moore, "Cramming More Components onto Integrated Circuits," *Electronics*, vol. 38, no. 8, pp. 114–117, 1965.
4. R. M. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, 1967.
5. H. Sutter, "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software," *Dr. Dobb's Journal*, vol. 30, no. 3, 2005.
6. D. Patterson, "Latency Lags Bandwidth," *Communications of the ACM*, vol. 47, no. 10, pp. 71–75, 2004.
7. N. P. Jouppi et al., "Ten Lessons from Three Generations Shaped Google's TPUv4i," *Proc. ISCA*, 2021 —

source for the energy cost data in Table (Section 4.7).
8. Lecture slides: L. Tornatore, "Modern Architecture," Foundations of HPC, UniTs, 2025/2026.
9. Memory latency data: PhD dissertation, "Memory Bandwidth and Latency in HPC: System Requirements and Performance Impact," Dept. of Computing Architectures, UPC Barcelona, 2019.

These notes are part of the Foundations of HPC course at Università di Trieste (2025/2026).
They have been written relying on the transcript from the lectures, re-touched using Claude opus 4.6.
Figures referenced in the text are from the lecture slides (01Modern_architecture.pdf) and from Hennessy & Patterson (7th ed., 2025). Students are encouraged to consult both the slides and the textbook for the visual material that accompanies these written notes.